Classwork:

Byte Pair Encoding Algorithm

hug → [h][u][g]
hugs → [h][u][g][s]
bug → [b][u][g]
bugs → [b][u][g][s]

⇒ Vocabulary: [h][u][g][s][b]

Count Frequency (of most occuring pair)

[h][u] → 2.
[u][g] → 4
[g][s] → 2
[b][u] → 2.

⇒ vocab grows: [h][u][g][s][b][ug]

new token.

hug → [h][ug]
hugs → [h][ug][s]
bug → [b][ug]
bugs → [b][ug][s]

⇒ vocab grows: [h][u][g][s][b][ug]

Count Freq:

[h][ug] → 2
[ug][s] → 2
[b][ug] → 2

when tied; pick randomly.

⇒ [h][u][g][s][b][ug][hug]

new token

hug → [hug]
hugs → [hug][s]
bug → [b][ug]
bugs → [b][ug][s]

Count Freq:

[hug][s] → 1
[ug][s] → 1
[b][ug] → 2

[h][u][g][s][b][ug][hug][bug]

new token.

use to embed.

hug → [hug]
hugs → [hug][s]
bug → [bug]
bugs → [bug][s]

stop here!

Side Note

BPE preserves some info:

Walked: [walk][ed]
Talked: [talk][ed]
Byzked: [by][z][k][ed]

suffix -ed tells us it is in past form.

Rule Based POS Tagger :

"text"
Input ⇒ (assign all possible tags) Tag Dictionary or Lexicon ⟹ Handwritten/manually defined Rules.

Doesn't work for words whose POS varies by context.

More than one possible POS!! called ambiguity! of tags.

book: VERB : Book that flight.
NOUN : Hand me that book.

i.e. An ambigious word is noun rather than verb if it follows a determiner.

- My friends play football every Saturday.
- I want to take part in the play.

because of determiner. noun

Correct Output Tag. (word, Tag)

Stochastic POS Tagger :

Two Primary Techniques

Word Frequency Measurement

The tag encountered most frequently in the training set is the one assigned to the ambigious instance of the word. with more than one possible POS.

also called (n-gram)
Tag Sequence Probabilities

The best tag for a given word is determined by the probability that it occurs with the n-previous tag.

noun verb determiner.
N V D ?
— — — ?

Q: what's the tag in this 4-gram sequence.
A: Compute the probability based on occurance of that sequence in the training sets!

Why POS Tagging?

1) Parsing → can improve syntactic parsing.
2) Sentiment → we may want to distinguish adjectives or other POS (that convey sentiment)
   can express intensity of any emotion.
3) Text-to-Speech → Some words are pronounced differently when used as "noun" vs "verb".
   LEAD → Material : Lead (used in industry)
   LEAD → Verb : (to take charge).
4) Helps with word order in a sentence.

Transformation-based tagging (TBL)

→ Introduced by Eric Brill in 1992

→ Objective: Develops a rule-based model that iteratively refines an initial state through transformations.

Initial State ⟹ **Transformations**

{assign naive label; i.e. most frequent tag}

{Rules that correct the errors in current tagging based on context}

Learning Process

- Identify errors in current tagging.
- Create a transformation rule to correct these errors
- Apply the rule & iterate.

Final Output

{a sequence of rules applied to improve tagging accuracy.}

Features of Transformation-based Tagging:

→ Combines Rule based & Stochastic Tagging.

- Rule based: Uses rules to specify tags in a certain environment.
- Stochastic approach: Utilizes a tagged corpus to find the best-performing rules.

→ Key Features: Rules are learned from data.

→ Input:

- Tagged Corpus: Provides the training data for learning rules.
- Dictionary: Contains the most frequent tags associated with words.

→ Limitations:

Quality of POS Tags generated depends on "Quality of the Rules"

) Use unicode for encoding!

[ord(x) for x in "कखग abc ☺"]

gives unicode for character!

[chr(x) for x in [11, 22, 33, 44]]

gives character for unicode.

Problem : 1) Keeps changing (constantly new chars added!)
2) Sequence becomes too large.

2) Try other versions. (utf-8, utf-16, utf-32)

utf-8, utf-16 → variable length output

utf-32 → fixed length output

"कखग abc ☺".encode("utf-8")

↳ print it as list to get raw values (otherwise you get byte stream)

Unicode Encoding of "म".

1) Character : म
also called unicode code point.

2) Unicode : U+092E (Unicode codepoints)

3) Decimal : 2350

4) Binary : 0000 1001 0010 1110 (16 bits)

↓ Follow UTF-8 encoding scheme

5) Final UTF-8 Encoding : 11100000 10100100 10101110 (Binary)

E0 Byte1 A4 Byte2 AE Byte3 (Hexadecimal)

Bytestream generated for म

6) Integer Representation. [224, 164, 174]

∴ "म" changes to [224, 164, 174] after utf-8 encoding.

⇓

IMP NOTE

⇒ In tiktoken Encoder. BPE (Byte Pair Encoding) works on these encoded bytestreams & merges them.

⇒ Sentencepiece Encoder runs BPE directly on unicode codepoints before even converting them to some form of encoded bytestreams

One Hot Encoding Problems:

1) Sparse Representation ; so wasteful!
2) Fails to capture semantic similarity. (good, nice may have very different represent
3) Out of vocabulary word will fail. (use OOV tokens)
4) No Fixed length representation for a sentence if flattened. Needs some sort of padding.

all vocabulary list

	add,	assist,	awkward,	awe,	are,	big,	can,	you	....	until,	yap,	OOV
Can (v1)	0	0	0	0	0	0	1	0				0
you (v2)	0	.	.	.	.	.	0	0	1	0	.	0
assist (v3)	0	1	0	.	.	.	.	.	.	.	.	0
until (v4)	0	0	.	.	.	.	.	.	0	1	0	.
tomorrow (v5)	0	0	.	.	.	.	.	.	.	.	0	1
please (v6)	0	0	0	.	.	.	.	.	.	.	0	1

(our sentence that needs to be encoded)

Can you assist until tomorrow please?

[v1] + [v2] + [v3] [v4] + [v5] + [v6] } variable length issue

Count Vectorizer creates this vector!

Bag of words Representation:

(word frequency count)

	inflation	can	iphone	you	android	ev	stock	growth	asset	jobs	assist
Sentence 1 (about job)	3	0	0	0	0	1	4	5	2	3	0
Sentence 2 (about phone)	1	0	2	0	3	0	0	1	0	0	0
Sentence 3 (about apple)	1	0	4	0	1	1	1	2	0	0	0
Can you assist until tomorrow please?	0	1	0	1	0	0	0	0	0	0	1

total vocabulary

vector size = size of vocab.

Note: Better than one-hot encoding. In one-hot encoding a single word would give a vector size of vocab. In count vectorizer, whole email gives that vector.

Disadvantages of count vectorizer: (Bag of Words)

1) Sparse representation
2) Can't capture relationship between similar sentences.

Training Steps:

Messages	Category	label
1) Offer! Offer! win . . .	spam	1
2) I think the world should understand . .	not spam	0
3) Nigerian prince has left his will .	spam	1

										label
[1	0	1	1	0	2	1]			→	1
[0	0	1	0	1	2	0	1	2]	→	0
[2	0	0	1	1	2	1	0	1]	→	1

TF-IDF:

terms(t)

Documents	musk	that	price	market	investor	iphone	ios	gigafactory . .
		(TF - unnormalized)						
Apple Article 1	[0	32	45	48	26	7	3	0 . . .]
IDF(t) →	[0	0.12	0.12	0.12	0.12	0.3	0.3	0 . . .]
Apple Article 2	[0	4	3	7	8	6	3	0 . . .]
Tesla Article 3	[15	31	44	43	25	0	0	0 . . .]
Tesla Article 4	[3	0	0	0	0	0	0	1 . . .]

Document Frequency (DF) = Number of times **word/term(t)** is present in **all docs**

terms	occurance in # DOCS
that →	3
gigafactory →	1
iphone →	2

$$IDF = \log\left(\frac{\text{Total Documents}}{\text{Number of documents term (t) is present in.}}\right)$$

Computing IDF for:

that → $\frac{4}{3} = 1.33$

gigafactory → $\frac{4}{1} = 4.$

iphone → $\frac{4}{2} = 2.$

the lesser a term occurs in # of Documents, higher the IDF

Term Frequency (TF)

$$TF(t,d) = \left(\frac{\text{Total No of times term } t \text{ is present in docA}}{\text{Total No of tokens in docA}} \right)$$

done to account for different document size

Why use log in IDF?

$$\boxed{TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)}$$

Let's say:

			count
DOC 1 →	word	computer	10
DOC2 →	word	computer	20

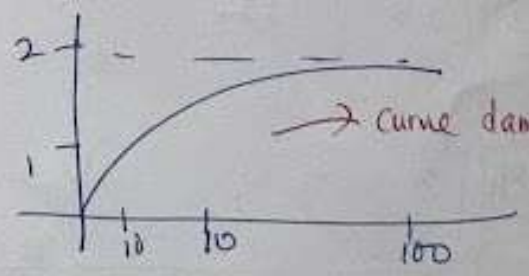
implies DOC 2 is more relevant for word computer.

However:

		count
DOC1 →	computer	1 million
DOC 2 →	computer	2 million.

both contain very high number; so both have relevance, although 2 million is still higher.

We take log to reduce that effect.



Limitations:

1) As no of words increase, sparsity increases.
2) Doesn't capture relation between words.

Perplexity: (measure of complexity in a sample of text)

> Perplexity is used to tell us whether a set of sentences look like they were written by humans rather than by a simple program choosing words at random. > A text written by humans is more likely to have a lower perplexity score.

> A text generated by random word choice will have a higher perplexity score.

$$PP(W) = \underbrace{P(s_1, s_2, \ldots, s_m)}_{\text{probability assigned by model to the sequence.}}^{-1/m}$$

where,

$W \rightarrow$ test set containing m sentences s.

$s_i \rightarrow$ i-th sentence in the test set, each ending with </s>

$m \rightarrow$ number of all words in entire test set W including </s> but not including <s>.

FORMAL DEFINITION:

> Perplexity is a key metric in NLP used to exaluate how well a language model predicts a sequence of words.

> It measures the model's uncertainty or "surprise" when encountering new text, with lower values indicating better predictive performance.

Interpretation:

> A perplexity of 1 means perfect prediction (no uncertainty), while higher scores - like 100 - suggest the model is as uncertain as randomly choosing among 100 words.

Confusion Matrix:

Predicted \ Actual	1	0
1	(TP)	(FP)
0	(FN)	(TN)

Precision:

Definition: Precision is the proportion of retrieved things that are relevant.

$$\boxed{\text{Pre}}\text{cision} = \frac{\text{True Positive}}{\boxed{\text{Pre}}\text{dicted Positive}} = \frac{TP}{\underbrace{(TP + FP)}_{\text{Hint: Predicted part is always written in second position}}}$$

Hint

In Example of "Cat" vs "Dogs" → Precision is the fraction of actual cats among the cats the ML system found (thought was cat)

Recall:

Definition: Recall is the proportion of relevant things that are retrieved.

$$\boxed{\text{Re}}\text{call} = \frac{\text{True Positive}}{\boxed{\text{Re}}\text{al Positive}} = \frac{TP}{(TP + FN)}$$

Hint

Example → Recall is the proportion of cats the ML system found correctly among all the cats in the images.

Precision vs Recall:

(books example)

→ A system with high precision might leave some good items out, but what it returns is of high quality.

(robbery example)

→ A system with high recall might give you lots of unwanted items, but it also returns most of the good items.

> $\boxed{\text{F-score measure}}$ is a way of balancing "Precision vs Recall".
harmonic mean between them.

Accuracy:

$$\boxed{\text{Accuracy} = \frac{TP+TN}{\text{Total}}}$$

Error Rate:

$$\text{Error Rate} = \boxed{1-\text{accuracy}}$$

OR

$$\boxed{\frac{FP+FN}{\text{Total}}} \approx \boxed{\frac{\text{Type 1 Error} + \text{Type 2 Error}}{\text{Total}}}$$

NOTE:

→ False Positive is also known as Type 1 Error.

→ False Negative is also known as Type 2 Error.

F1-score:

If Class is imbalanced; $\left\{\begin{array}{l}\text{Basic}\\ \text{Accuracy}\\ \text{measures}\end{array}\right\}$ will not give a clear picture.

$$\boxed{\text{F1-score}} = \frac{2}{\frac{1}{P}+\frac{1}{R}} = 2\times\left(\frac{PR}{P+R}\right)$$

OR,

$$\boxed{\text{F1-score} = \frac{TP}{TP+\frac{1}{2}(FP+FN)}}$$

NOTE:

For Numbers: 1, 2, 3, 4, 5

$$\text{Harmonic mean (HM)} = \frac{5}{\underbrace{\frac{1}{1}+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{5}}_{\text{reciprocal of individual values}}}$$

5 } total item count